SQL_TEST_01 · COMMAND REFERENCE · LIVING DOCUMENT

Everything you've typed so far

Every command, meta-command, and function actually used hands-on across this project — grouped by tool, in the order you met them, not alphabetized — plus a project-wide file map, and a running list of Claude's own working commands. Appended at the end of each build, not written once and frozen.

COVERS

Build 1 (agent) through Build 6 (front end for the agent) — all complete

ENVIRONMENT

Windows 11 · PowerShell · Docker Desktop · GitHub Actions

DATABASES

pg_local (appdb)

Project Files · Docker · psql · PowerShell · Git · GitHub CLI · psycopg · psycopg.sql · pandas · pandera · pytest · pgvector & embeddings · BeautifulSoup · Python stdlib · Streamlit · Claude's Commands

Project Files

WHAT'S ACTUALLY IN THIS FOLDER

A map of the project root as of Build 5's completion — what each file is for, not what commands it contains (that's every section below). Excludes auto-generated folders (`.venv/`, `__pycache__/`, `.git/`).

SQL migrations — run in order, numbered for reproducibility

`001_schema.sql`

Original appdb schema — `customers`/`orders`, the seed data every later build extends.

`002_seed.sql`

Idempotent seed data for `customers`/`orders`.

`003_readonly_role.sql`

`appdb_reader` — the read-only role Build 1's agent runs as.

`004_raw_schema.sql`

Build 2 Phase 1 — `raw.allocations`, all-`TEXT` ingestion target.

`005_clean_schema.sql`

Build 2 Phase 2 — `clean.allocations`, real types plus the computed `duration_seconds` column.

`006_agent_scratch_role.sql`

Build 2.5 Phase 1 — `appdb_agent_writer`, the agent's write-scoped role.

`007_agent_scratch_schema.sql`

Build 2.5 Phase 1 — the `agent_scratch` schema and its grants.

`008_add_summary_embedding.sql`

Build 3 Phase 1 — adds `clean.allocations.summary_embedding vector(384)`.

`009_add_summary_embedding_hnsw_index.sql`

Build 3 follow-up — HNSW index on `summary_embedding` for indexed similarity search.

`010_docs_chunks_schema.sql`

Build 3.5 Phase 1 — the `docs.chunks` table (`vector(384)`) for the documentation RAG corpus.

`011_metrics_history_schema.sql`

Build 4 follow-up — `observability.metrics_history`, a tracked time series over `metrics_rollup.py`'s runs.

`012_chat_rounds_schema.sql`

Build 6 Phase 3.5 — `agent_scratch.chat_rounds`, persistent shared chat history surviving a real container restart.

`schema_mssql.sql`

The SQL Server counterpart to `001_schema.sql` — same appdb shape, T-SQL dialect.

`seed_mssql.sql`

SQL Server counterpart to `002_seed.sql`.

Python — the agent itself

`agent.py`

The hand-built tool-use loop, wrapped in a callable `ask_agent()` since Build 4 — calls the Anthropic API, dispatches tool calls, enforces `MAX_TOOL_TURNS`, tags every logged call with a `question_id`.

`tools.py`

Every tool the agent can call: `run_sql_query`, `list_tables`, `describe_table`, `save_dataframe`, `search_summaries`, `search_docs`.

Python — observability & eval harness (Build 4)

`logging_utils.py`

Structured JSON Lines tool-call logging — one line per call: timestamp, `question_id`, turn, tool name, input, output, error, latency.

`eval_cases.py / eval_cases_round2.py`

Fixed sets of test questions with an expected tool and expected answer keywords — run end-to-end against the live agent, not tested in isolation.

`run_eval.py`

Drives every case through `ask_agent()`, checks the tool-call log and answer text, reports pass/fail per question. Accepts an optional case-module argument.

`metrics_rollup.py`

Plain-text report over the tool-call log — tool-usage frequency, average turns per question (grouped by `question_id`), error rate.

Python — live deployment with CI (Build 5)

`synthesize_dataset.py`

Builds `data/ALLOCATION-synthesized.csv` from the real export — real dates/duration/renewal/resource counts kept, department/email/summary synthesized. Imports the real department-code mapping from a git-ignored local file, never hardcodes it.

`department_mapping.local.py`

Git-ignored — the real department-code mapping `synthesize_dataset.py` actually runs from. Never committed; see `DEPARTMENT_MAPPING.local.md` for the human-readable copy.

`pytest.ini`

`python_files = test_*.py` — scopes pytest discovery to this project's actual test-file convention, after a stray non-test script (`api_test.py`) was found being silently collected by pytest's broader default.

Python — front end for the agent (Build 6)

`app.py`

The Streamlit entrypoint — chat UI wrapping `ask_agent()`, hybrid tiered history threading via `session_state.rounds`, a persisted feed (Phase 3.5) loaded from `agent_scratch.chat_rounds` on first load, and an FAQ-style render: only the newest round shows expanded, older rounds collapse into `st.expander(label=the question)`.

`test_agent_history.py`

3 tests for `flatten_history()` — the pure function behind hybrid tiered conversation memory.

`012_chat_rounds_schema.sql`

Build 6 Phase 3.5 — `agent_scratch.chat_rounds`, the durable store behind the persisted chat feed. Reuses `appdb_agent_writer`, no new role. Grants `SELECT, INSERT, DELETE` — `DELETE` added after the fact once `MAX_CHAT_ROUNDS` needed it.

`test_chat_rounds.py`

3 tests — save/load round-trip, discovery exclusion, and `MAX_CHAT_ROUNDS`'s exact boundary via `monkeypatch`.

Python — data pipeline (Build 2)

`ingest.py`

Loads the source CSV into `raw.allocations`, one `executemany` call.

`profile_raw_data.py`

Pandas exploration of `raw.allocations` that informed `005_clean_schema.sql`'s design. Renamed from `profile.py` — it silently shadowed Python's stdlib `profile` module.

`transform.py`

Raw → clean transform: type coercion, blank→NULL handling, populates `clean.allocations`.

Python — semantic search (Build 3)

`embed_summaries.py`

Idempotent backfill — embeds every `summary` row still missing a `summary_embedding`.

`search_summaries.py`

Standalone script proving raw cosine-distance search works, before any agent wiring.

`calibrate_threshold.py`

Statistical calibration of the 0.63 relevance-distance cutoff, from 40 curated in-/out-of-domain queries.

Python — documentation RAG (Build 3.5)

`extract_doc_chunks.py`

Parses this project's own docs into embeddable chunks — four extraction shapes across two design eras, plus narrative prose. `SOURCE_FILES` auto-discovers every handoff brief via `glob` (Build 6 fix) rather than a hand-maintained list, after that list twice silently fell behind a newly-written brief.

`embed_chunks.py`

Idempotent backfill for `docs.chunks` — same `WHERE embedding IS NULL` pattern as `embed_summaries.py`.

`search_chunks.py`

Standalone script proving raw similarity search over the docs corpus works, before any agent wiring.

Python — early sanity checks (pre-Build-1)

`db_test_py.py`

The very first Postgres connectivity check — plain `psycopg`, no agent involved.

Tests

`test_data_quality.py`

Build 2 — `pandera` schema checks against `clean.allocations`.

`test_agent_scratch_boundary.py`

Build 2.5 — role-boundary tests plus the `MAX_SCRATCH_TABLES` cap test.

`test_semantic_search.py`

Build 3 — embedding coverage, relevant/irrelevant query behavior, HNSW index existence.

`test_docs_search.py`

Build 3.5 — same shape as `test_semantic_search.py`, applied to the documentation corpus.

Configuration & infrastructure

`docker-compose.yml`

Defines `pg_local`, `mssql_local`, the `agent` service, and (Build 6) `streamlit_app` — same image as `agent`, just a different `command:`. Postgres image is `pgvector/pgvector:pg17`.

`Dockerfile`

Builds one shared image used by both the `agent` and `streamlit_app` services — differentiated only by each service's `command:` override, not two separate Dockerfiles.

`requirements.txt`

Pinned Python dependencies — `anthropic`, `beautifulsoup4`, `psycopg`, `pgvector`, `sentence-transformers`, `pandas`, `pandera`, `pytest`, `python-dotenv`, `streamlit` (Build 6).

`.env`

Real local secrets (DB passwords) — gitignored, never committed.

`.env.example`

Template shape of `.env` with placeholder values, safe to commit.

`.gitignore`

Excludes secrets (`.env`, `CREDENTIALS.local.md`, `DEPARTMENT_MAPPING.local.md`, `department_mapping.local.py`), OS/editor cruft, `__pycache__/`, logs/dumps.

`.github/workflows/ci.yml`

Build 5 — GitHub Actions: migrations + full pipeline + pytest on every push; a gated eval-harness job on a relevant merge to `main` only.

`.gitattributes`

Forces consistent line endings for `.sql/.yml/.md` across Windows/Unix.

Docs, secrets & generated artifacts

`CREDENTIALS.local.md`

Unredacted local companion to `.env` — gitignored, never shared or committed.

`DEPARTMENT_MAPPING.local.md`

Build 5 — the real department-code mapping used by `synthesize_dataset.py`, human-readable. Gitignored, same treatment as `CREDENTIALS.local.md` — publishing it would let anyone reverse the department masking.

`data/ALLOCATION-synthesized.csv`

Build 5 — the project's one official dataset (1535 rows), real temporal/quantitative fields kept, everything else synthesized. Used identically by local dev, CI, and the eval harness.

`sql-test-01-cheatsheet.html / .pdf`

This living document — every command actually typed, plus this file map.

`PROJECT_DIAGRAMS/`

One subfolder per build (`BUILD_1_FLOW` … `BUILD_6_FLOW`) — flow diagrams (`.svg/.html/.png`) and handoff briefs (`.html/.pdf`). Build 1's two handoff briefs live here too, recovered from Artifacts during Build 3.5.

`PROJECT_DIAGRAMS/project-overview.html`

Build 3.5 — a seed document written to answer whole-project overview questions, so the docs-RAG corpus has something to match against.

`logs/`

Build 4 — gitignored directory holding `tool_calls.jsonl` (live, JSON Lines) and `tool_calls.pre-question-id.jsonl` (archived). Durable, queryable tool-call tracing — not a Postgres table, deliberately.

`Local Postgres.session.sql`

Empty scratch file created by a VS Code SQL client extension — not authored content.

`.vscode/, .pytest_cache/`

Editor settings and pytest's own cache — auto-generated, not source.

Docker

CONTAINER ORCHESTRATION

Starting the stack, and getting SQL in and out of a running container.

```
docker compose up -d --build
```

Build images if anything changed, then start every service in the background.

```
docker ps
```

List running containers — the first check when a connection unexpectedly fails.

```
docker exec -it pg_local psql -U devuser -d
appdb
```

Open a **live, interactive** psql session inside the Postgres container.

```
Get-Content file.sql -Raw | docker exec -i
pg_local psql -U devuser -d appdb
```

Run a whole .sql file inside the container **non-interactively**, PowerShell-style — < redirection doesn't exist in PowerShell, so the file gets piped in instead. (Bash/cmd would use `docker exec -i ... < file.sql`.)

```
docker exec pg_local psql -U devuser -d appdb
-c "SELECT ..."
```

Run a single one-off SQL command non-interactively, no piped file needed. **No `-it`** — that flag requires a real terminal attached to stdin and fails with `"cannot attach stdin to a TTY-enabled container"` in non-interactive contexts (e.g. run from a script/tool rather than a live shell).

```
docker compose down -v
```

Stop everything and delete volumes too — next start re-initializes DB passwords from .env. Used deliberately, not casually.

Build 6 — `COPY` bakes source in at build time, with no hot-reload path at all. Editing `agent.py`/`tools.py`/`app.py` on the host has zero effect on an already-running container, no matter what — stricter than the local Streamlit dev-server case, which at least auto-reruns `app.py` on save. A real `docker compose build` + `up` (recreating the container) is required every time, confirmed directly via `docker exec ... grep` against the container's own copy of a file rather than trusting the build log alone.

psql

META-COMMANDS & DDL PATTERNS

Once you're inside a psql session — both the backslash meta-commands and the recurring SQL shapes.

```
\d table_name
```

Describe a table — every column, its type, and any constraints. The one check that settles "did my CREATE TABLE actually work."

```
\du role_name
```

Describe a role — confirms it has no Superuser/Createrole/Createdb it shouldn't.

```
\pset null 'marker'
```

Display NULLs as a visible marker instead of blank — otherwise a NULL and an empty string look identical.

```
\! cls
```

Shell out and run an OS command without leaving psql — `cls` clears the terminal screen.

```
\q
```

Quit psql, back to the shell prompt.

```
CREATE SCHEMA IF NOT EXISTS name;
```

Create a schema, skip quietly if it's already there — safe to rerun.

```
DROP TABLE IF EXISTS name;
```

Drop a table, skip quietly if it doesn't exist — what makes a rebuild script rerunnable from a blank database.

```
GRANT SELECT ON ALL TABLES IN SCHEMA s TO
role;
```

Give a role read access to every table **currently** in a schema.

```
ALTER DEFAULT PRIVILEGES IN SCHEMA s GRANT
SELECT ON TABLES TO role;
```

Extend that same grant automatically to any table created **later** — without this, new tables start private.

```
col TYPE GENERATED ALWAYS AS (expr) STORED
```

A column Postgres computes and locks from other columns in the same row — structurally impossible for it to drift out of sync.

```
GRANT USAGE, CREATE ON SCHEMA s TO role;
```

Let a role create objects inside a schema, not just read from it — how `appdb_agent_writer` got write access scoped to `agent_scratch` only, never anywhere wider.

```
ALTER DEFAULT PRIVILEGES FOR ROLE owner IN
SCHEMA s GRANT SELECT ON TABLES TO role;
```

Extend a **different** role's read access to tables a specific owner role creates later in that schema — how `appdb_reader` can always see whatever `appdb_agent_writer` saves, without a re-grant each time.

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Enable pgvector for the database — requires the `pgvector/pgvector` Postgres image, not the plain `postgres` one.

```
ALTER DATABASE db REFRESH COLLATION VERSION;
```

Clear the collation-version mismatch warning that appears after swapping to an image built against a different glibc/ICU — safe here since the data is plain ASCII text.

```
col vector(384)
```

pgvector's column type for a fixed-dimension embedding — 384 matches the sentence-transformers model in use, not an arbitrary choice.

```
a ⟺ b
```

Cosine distance between two vectors — smaller means more similar. What `search_summaries` orders by.

```
a ⟷ b
```

Euclidean (L2) distance between two vectors — used in this project only for the self-distance sanity check (`v <-> v` should be exactly 0).

```
vector_dims(col)
```

The dimensionality of a stored vector — a quick sanity check that every row actually has a real 384-dim embedding, not a malformed one.

```
CREATE INDEX ... USING hnsw (col
vector_cosine_ops)
```

Build an approximate-nearest-neighbor graph index matching the `<=>` operator, so similarity search can use an index scan instead of a full table scan as the corpus grows.

```
EXPLAIN ANALYZE SELECT ...
```

Confirms whether the planner actually used a new index (`Index Scan`) or fell back to `Seq Scan` — the only real way to verify an index is doing anything.

PowerShell

HOST-SIDE SHELL

Running Python, checking your own environment, and vetting/installing system-level tools from outside any container.

```
.\.venv\Scripts\python.exe script.py
```

Run a script with the project's own virtual-env interpreter directly — sidesteps PATH/activation issues entirely.

```
.\.venv\Scripts\python.exe -m pip install pkg
```

Install a package into that specific virtual env, not wherever PATH happens to point.

```
.\.venv\Scripts\python.exe -m pip show pkg
```

Check whether a package is installed, and exactly which version.

```
Get-ChildItem Env: | Where-Object { $_.Name -
like "*X*" }
```

List environment variables whose name matches a pattern — e.g. catching a stray API key leaked into the shell.

```
Get-Content file
```

Print a file's contents — the go-to way to verify what actually got saved to disk.

```
claude auth status
```

Confirm this Claude Code session is on subscription login, not silently metered API billing.

```
winget show --id pkg -e
```

Pull a package's full manifest — publisher, source URL, license, install hash — before trusting or installing it.

```
winget install --id pkg -e
```

Install a package system-wide once it's been vetted, not before.

Git

CHECKPOINTING PROGRESS

Committing at the end of every phase, per the project's own working style.

`git status`

What's changed, staged, or untracked, right now.

`git add file`

Stage a specific file by name — never a blind `-A`.

`git commit -m "..."`

Record the staged snapshot with a message explaining **why**, not just what.

`git mv old new`

Rename a tracked file while git keeps its history attached to the new name.

`git log --oneline -n`

Recent commit history, one line each — the fastest way to check "where are we, really."

`git diff`

Exact unstaged changes, line by line, before they get staged.

`git remote add origin url / git push --set-upstream origin main`

Build 5 — wire up a GitHub remote for the first time and push with tracking, so plain `git push` works afterward.

`git stash push -u -m "..." / git stash pop`

Build 5 — shelve in-progress uncommitted work (including untracked files, via `-u`) to get a clean tree before a history rewrite, then restore it after.

`git commit --amend`

Build 5 — rewrite the tip commit in place. Safe only when that commit was **never pushed**; used to fix a real mistake (real data hardcoded in a script) before it ever left the machine.

`git bundle create out.bundle --all / git bundle verify out.bundle`

Build 5 — a full, portable backup of every ref in the repo in one file. Taken before every `git filter-repo` pass this build, saved outside the repo itself.

`git log --all -S"string" --oneline`

Build 5 — pickaxe search: commits where a literal string's occurrence count changed. The actual verification method used to confirm a purge worked, not assumed from the tool's own success message.

`git show <commit>:<path>`

Build 5 — print a file's exact content as of one specific commit, without checking it out — used to directly confirm a blob no longer contains something it used to.

```
git reflog expire --expire=now --all && git gc
--prune=now
```

Build 5 — force-expire the reflog and immediately garbage-collect, so an amended-away commit's old blob is actually gone locally, not just unreferenced.

```
git push --force origin main
```

Build 5 — overwrite the remote's history after a `filter-repo` rewrite. Only used after explicit confirmation, and only because the rewrite genuinely changed already-pushed commits.

GitHub CLI

GH · BUILD 5

Installed via `winget` mid-build (not present by default) — authentication and repo creation still required the user directly; everything past that ran from this session.

```
gh auth login / gh auth status
```

Authenticate gh to a GitHub account (interactive — requires the user's own browser/credentials) and confirm the current login, protocol, and token scopes.

```
gh repo create name --public --source=. --
remote=origin --push
```

Create a new GitHub repo from the current local repo and push it in one step — the intended one-liner, not usable here since `gh` wasn't installed yet at that point.

```
gh secret set NAME --repo owner/repo
```

Set a GitHub Actions secret, prompting for the value securely (never echoed, never logged) — how `ANTHROPIC_API_KEY` got configured, by the user directly.

```
gh run list --limit n
```

Recent Actions runs for the current repo — status, workflow, branch, duration.

```
gh run view <id>
```

Step-by-step breakdown of one run — which steps passed, which failed, without opening a browser.

```
gh run view <id> --log-failed
```

Full log output for only the failed step(s) — the actual diagnostic step every real CI failure this build was root-caused from.

```
gh run watch <id>
```

Streams a run's live progress until it completes — used after every push this build to confirm a fix before moving on, rather than polling manually.

Python — psycopg

TALKING TO POSTGRES

The driver behind every script in this project — agent.py, tools.py, ingest.py, profile.py.

```
psycopg.connect(host=, dbname=, user=,
password=)
```

Open a connection to a Postgres database.

```
with conn.cursor() as cur:
```

Open a cursor scoped to a `with` block — closes itself automatically, even on error.

```
cur.execute(sql, params)
```

Run one SQL statement, with parameters passed safely (never string-formatted into the query).

```
cur.executemany(sql, rows)
```

Run the same statement once per row — how ingest.py loads 1,535 rows in one call.

```
cur.fetchall()
```

Pull every result row back as a Python list.

```
cur.description
```

Metadata about the result's columns — used to grab column names generically.

```
conn.commit()
```

Make written changes permanent.

```
conn.close()
```

Release the connection.

Python — psycopg.sql

SAFE DYNAMIC IDENTIFIERS

save_dataframe's guardrail for building a CREATE TABLE/INSERT statement where the table and column names themselves come from the model, not just the values — %s parameters alone can't protect an identifier.

`from psycopg import sql`

The submodule that builds SQL safely out of parts psycopg can't treat as plain data — like table and column names.

`sql.Identifier(name)`

Safely quotes a dynamic identifier (table/column name) — the equivalent of a parameter, but for names instead of values.

`sql.SQL(" … {} … ").format( … )`

Builds a SQL string from literal fragments plus safely-escaped pieces like `Identifier` — never plain `f"..."` string interpolation.

`sql.SQL(", ").join(parts)`

Joins a list of `Identifier`/`SQL` fragments with a separator — how a variable-length column list gets assembled.

`sql.Placeholder()`

A %s value placeholder built the same composable way, for when the number of columns isn't known until runtime.

Python — pandas

PROFILING RAW DATA

Everything profile_raw_data.py used to understand raw.allocations before designing the clean schema. (Renamed from profile.py in Build 3 — it was silently shadowing Python's stdlib profile module.)

`pd.DataFrame(rows, columns=cols)`

Build a DataFrame straight from plain rows + column names — no SQLAlchemy required.

`df["col"].unique()`

Every distinct value in a column, once each — printed with `repr()` to expose hidden whitespace.

`df["col"].value_counts(dropna=False)`

Count of each distinct value, including blanks/NaN.

`pd.to_datetime(s, format=, errors="coerce")`

Parse a column of date strings against an exact format; anything that doesn't match becomes `NaT` instead of crashing the run.

`series.dt.total_seconds()`

Convert a time difference into a plain number of seconds.

`series.str.fullmatch(pattern)`

Test every value in a column against a regex at once — used to confirm two columns were digit-only.

Python — pandera

DATA-QUALITY SCHEMAS

test_data_quality.py uses this to declare what "correct" looks like for clean.allocations, as data instead of ad hoc checks.

`import pandera.pandas as pa`

The explicit pandas-backend import for modern pandera versions.

`pa.DataFrameSchema({ ... })`

Declare an entire DataFrame's expected shape — one entry per column — as data, not scattered asserts.

`pa.Column(type, checks, nullable=)`

One column's expected type, whether NULLs are allowed, and any constraints.

`pa.Check.ge(0)`

A reusable constraint — here, "greater than or equal to 0" — attached to a Column.

`schema.validate(df)`

Check a real DataFrame against the schema; raises a real error the moment something doesn't match.

pytest

RUNNING THE TEST SUITE

First automated test suite in the project — codifies checks that were previously done by hand.

`.\.venv\Scripts\python.exe -m pytest -v`

Discover and run every `test_*` function in the project, printing each one's name and pass/fail individually.

`def test_something():`

Any function named `test_...` in a `test_*.py` file is auto-discovered — no registration needed.

`assert condition`

Plain Python `assert` — pytest reports exactly which assertion failed and the actual vs. expected values.

`with pytest.raises(ExceptionType):`

Asserts a block **must** raise a specific exception — used to prove a role boundary actually blocks a write, not just that it "seems fine."

```
def test_x(monkeypatch):
monkeypatch.setattr(mod, "NAME", val)
```

Temporarily overrides a module-level constant for one test only, auto-restored after — how a cap like `MAX_SCRATCH_TABLES` gets tested at its boundary without creating 50 real tables.

pgvector & embeddings

SENTENCE-TRANSFORMERS · PGVECTOR.PSYCOPG

Turning free text into vectors and getting Postgres to store/compare them — `embed_summaries.py`, `search_summaries.py`, `tools.py`'s `search_summaries`.

```
from sentence_transformers import
SentenceTransformer
```

A local, free embedding model library — no API key or per-call cost, unlike Voyage AI/OpenAI embeddings.

```
SentenceTransformer("all-MiniLM-L6-v2")
```

Loads the specific 384-dim model used throughout this project — first call downloads and caches it locally.

```
model.encode(text_or_texts,
show_progress_bar=True)
```

Turns one string or a list of strings into embedding vector(s) — a progress bar matters when embedding 1,500+ rows at once.

```
from pgvector.psycopg import register_vector
```

Teaches a psycopg connection how to send/receive pgvector's `vector` type directly as numpy-like arrays, instead of raw strings.

```
register_vector(conn)
```

Applies that registration to a specific connection — needed once per connection, before any vector query runs.

```
cur.execute("... WHERE col ⟺ %s < %s ...",
(embedding, threshold))
```

Passes a computed embedding straight in as a query parameter — the distance operator and the threshold cutoff both happen inside the SQL, not in Python after fetching everything.

Python — BeautifulSoup

HTML PARSING & CHUNK EXTRACTION

Turning this project's own handoff briefs/cheat sheet into embeddable chunks — `extract_doc_chunks.py`, Build 3.5.

`from bs4 import BeautifulSoup`

The HTML parsing library — reads real markup instead of treating a doc as one flat blob of text.

`BeautifulSoup(html_text, "html.parser")`

Parses a string of HTML into a navigable tree, using Python's built-in parser (no extra system dependency like `lxml`).

`soup.select(".callout")`

CSS-selector search — returns every element matching a class/tag, the same selector syntax as the CSS already written for these docs.

`soup.select_one(".fn")`

Like `select`, but returns just the first match (or `None`) — used when a chunk has at most one label element.

`element.get_text(" ", strip=True)`

Pulls plain text out of an element and its children, joining pieces with a space and trimming whitespace — turns nested markup into one clean string to embed.

Python — standard library

CSV · OS · DOTENV

No extra install required — these ship with Python itself, or python-dotenv.

```
csv.DictReader(f)
```

Read a CSV where each row comes back as a `{column: value}` dict — chosen over pandas for ingestion specifically so nothing gets silently type-coerced.

```
open(path, newline="", encoding="utf-8-sig")
```

Open a file safely for CSV reading, transparently stripping a leading BOM if present.

```
os.environ["KEY"]
```

Read a required env var — raises immediately if it's missing, rather than continuing with a silent gap.

```
os.environ.get("KEY", default)
```

Read an optional env var, falling back to a default (e.g. `POSTGRES_HOST` defaulting to `127.0.0.1` outside Docker).

```
load_dotenv(encoding="utf-8-sig")
```

Load .env into the process environment — BOM-safe, matching how Windows tools often save that file.

```
json.dumps(entry, default=str)
```

Serialize a dict to a JSON string, falling back to `str()` for anything the default encoder can't handle (e.g. `datetime`) — how `logging_utils.py` keeps every logged line valid JSON regardless of what a tool result contains.

```
open(path, "a") as f:
f.write(json.dumps(entry) + "\n")
```

The JSON Lines pattern — one independent JSON object per line, appended, never rewriting the whole file. Read back with `json.loads(line)` per line, not one big `json.load()`.

```
uuid.uuid4().hex[:12]
```

A short random ID with no coordination needed across processes — how each `ask_agent()` call gets a distinct `question_id`. Not a counter: a shared counter would itself need synchronization once calls can run concurrently.

```
time.perf_counter()
```

A monotonic, high-resolution clock for measuring elapsed time — call before and after, subtract, multiply by 1000 for milliseconds. Not `time.time()`, which can jump if the system clock adjusts.

```
sys.argv[1] if len(sys.argv) > 1 else default
```

Read an optional CLI argument, falling back to a default — how `run_eval.py` picks which eval-case module to run.

```
importlib.import_module(name)
```

Import a module by its name as a string, decided at runtime — lets `run_eval.py` load `eval_cases` or `eval_cases_round2` (or any future round) with no if/elif import chain.

```
from collections import Counter;
Counter(items).most_common()
```

Count occurrences of each distinct value and list them ranked most-to-least frequent — `metrics_rollup.py`'s tool-usage frequency.

```
n = len(open(path).readlines()); ... ;
f.readlines()[n:]
```

Checkpoint-based tailing — count existing lines before an action, then re-read only the lines appended after. How `run_eval.py` isolates one `ask_agent()` call's new log entries without `ask_agent()` needing to return that data itself.

Streamlit

BUILD 6 · APP.PY

Everything `app.py` uses to turn `ask_agent()` into a real chat interface — session state, chat primitives, and the Phase 4 tool-call expander.

```
streamlit run app.py --server.address=0.0.0.0
```

Starts the dev server. The address flag is only needed inside Docker — Streamlit defaults to binding `localhost` only, unreachable through a container's port mapping otherwise; not needed for a plain local run.

`st.session_state`

A dict-like store that survives Streamlit's full-script rerun on every interaction — a plain Python variable would reset to its initial value every time. Holds live Python objects directly; no JSON serialization needed within one session.

```
if "key" not in st.session_state:
st.session_state.key = default
```

The standard init-once guard — runs on every rerun, but only actually sets the default the first time.

`st.chat_message(role)`

A styled message bubble for a given role (`"user"`/`"assistant"`) — used as a `with` block, contents render inside it.

`st.chat_input(placeholder)`

The bottom-anchored chat text box — returns the typed text once submitted, otherwise `None`.

`st.spinner(" ... ")`

Shows a loading indicator for the duration of a `with` block — wraps the `ask_agent()` call so the wait for a real API round-trip is visibly acknowledged.

`st.error(msg)`

A visually distinct red message box — used to surface `ask_agent()`'s error field separately from a normal answer, not swallowed.

`st.expander(label)`

A collapsible section, closed by default — the `Tools used (N)` block per answer, keeping tool-call detail available without it dominating the page.

`st.code(text, language="json")`

A syntax-highlighted, monospace code block — used for each tool call's formatted `input`.

`st.caption(text)`

Small, muted secondary text — the `PROCESS_ID` line under the title, and each tool call's latency.

Build 6 Phase 3.5 — `st.expander` cannot nest inside another `st.expander`. Hit while building the FAQ-style collapsed-round view (each older round wraps its answer in an expander labeled with the question) — a round's `Tools used` detail (itself normally an expander, Phase 4) has to render as plain inline text/captions instead whenever it's inside an already-collapsed round, and only get the real widget when that round is the one currently shown expanded.

Claude's Commands

DIAGRAMS · VERIFICATION · PUBLISHING

A separate running list from everything above — not what *you've* typed, but the commands Claude actually runs behind the scenes while building diagrams, verifying renders, and publishing docs for this project. Appended to as new patterns come up, same as every other section.

```
msedge --headless --disable-gpu --window-
size=W,H --screenshot=out.png
file:///path.html
```

Renders an HTML file exactly as a browser would and saves it as a PNG — the first of two required checks before any diagram or doc is considered verified.

```
msedge --headless --disable-gpu --print-to-
pdf=out.pdf --print-to-pdf-no-header --no-pdf-
header-footer file:///path.html
```

Renders the same page through Chromium's separate print pipeline — a genuinely different code path that has caught real bugs (`<img>` references silently failing to render) the screenshot path alone missed.

```
pdftoppm -png -r 100 file.pdf prefix
```

Rasterizes every page of a generated PDF into numbered PNGs, so each page can actually be looked at instead of trusting file size alone.

```
pdfinfo file.pdf | grep -i pages
```

Quick page-count check before rasterizing — confirms whether a doc paginated the way it was expected to.

```
git commit -m "$(cat <<'EOF'  ...  EOF)"
```

Heredoc commit-message convention — explains **why** a change happened in the body, and avoids hand-escaping quotes/newlines. Every commit message ends with a `Co-Authored-By` line.

```
Image.open(path).convert("RGB").getpixel((x,
y))
```

Samples an exact on-screen pixel's RGB value with Pillow — confirms a color rendered exactly as CSS specified, rather than trusting a visual impression at small/compressed scale. Caught a false alarm (anti-aliasing fringe mistaken for a real color bug) during this cheat sheet's own verification.

```
WebFetch(url="claude.ai/code/artifact/<uuid>",
 ... )
```

For Artifacts the user owns, this returns full raw HTML rather than the usual markdown conversion — used to recover Build 1's two handoff briefs, which had only ever been published as Artifacts, never saved as local files.

```
cygpath -m "$(pwd)/file.html"
```

Converts a Git Bash path to a Windows-style forward-slash path (`C:/...`) — necessary before building a `file:///` URL for headless Edge, since Git Bash's own `/c/...` path style isn't a valid Windows path and silently fails to load.

```
git log --oneline <hash>^..<hash>
```

The exact commit range between two specific commits (the leading `^` makes it inclusive of the first one) — used to pin down a build's full commit range for its handoff brief, rather than `-n`'s "last N commits."

```
pdftotext file.pdf -
```

Build 5 — extracts a PDF's actual text to stdout. Used specifically because a plain `grep` on a PDF's raw bytes can silently miss encoded/compressed text — the real check after redacting something from a rendered document.

> **Headless Edge sometimes needs a command run twice.** The first invocation can exit with code 2 and no output file at all; an identical second call then succeeds. Observed repeatedly across both screenshot and print-to-pdf renders — not project-specific, a Windows/Chromium quirk. Always confirm the output file actually exists (and, for PDFs, that `pdfinfo` reports the expected page count) after a render — never assume the first call worked.

Living document — appended at the end of each build, not written once and frozen. Reflects hands-on usage through Build 6, complete end to end including Phase 3.5 — a real Streamlit chat UI runs alongside the CLI agent, both as docker-compose services sharing one image, with chat history now surviving a real container restart. Not exhaustive documentation for any of these tools, just what's actually been typed in this project so far.